Phase 4: Presettable Program Counter Implementation

Objective

Replace the sequential-only binary counters with presettable up/down counters to enable jump, branch, and loop operations in the CPU.

Component BOM

New Components

Qty	Component	Function
4	SN74LS190N	4-bit up/down decade counters (16-bit PC)
2	74F374N	16-bit address latch (jump target storage)
1	74F08	AND gates (load control logic)
1	74F04N	Hex inverters (signal conditioning)
16	Toggle switches	Manual jump address input (testing)
2	Pushbuttons	Manual load enable, reset
4	0.1µF capacitors	Additional decoupling
1	Toggle switch	Up/Down control
-	Additional jumper wires	Counter chaining

Replaced Components

Remove	Replace With	Reason
2x SN74LS590N	4x SN74LS190N	Add preset capability

Counter Configuration

SN74LS190N Pin Functions

Pin 1-4: P0-P3 (parallel load inputs)
Pin 5: /PL (parallel load enable, active low)
Pin 6-7: GND, VCC
Pin 8: /MR (master reset, active low)
Pin 9: U/D (up/down control: low=down, high=up)
Pin 10: CP (clock pulse input)
Pin 11: TC (terminal count output)
Pin 12: /RC (ripple clock output, active low)
Pin 13-16: Q0-Q3 (counter outputs)

16-Bit Counter Chain

Counter 0 (LSB): Handles bits 0-3 of program counter
Counter 1: Handles bits 4-7 of program counter
Counter 2: Handles bits 8-11 of program counter
Counter 3 (MSB): Handles bits 12-15 of program counter

Chaining:
Counter 0 TC → Counter 1 CP (ripple carry)
Counter 1 TC → Counter 2 CP
Counter 2 TC → Counter 3 CP

Enhanced Instruction Set

Jump Instructions

Opcode	Mnemonic	Description	Implementation
1001	JMP addr	Unconditional jump	Load addr into PC
1010	JZ addr	Jump if zero flag set	Conditional load
1011	JC addr	Jump if carry flag set	Conditional load
1100	JNZ addr	Jump if zero flag clear	Conditional load
1101	JNC addr	Jump if carry flag clear	Conditional load

Multi-Byte Instruction Format

JMP instruction (3 bytes):
Byte 0: 0x9X (JMP opcode)
Byte 1: High byte of target address
Byte 2: Low byte of target address

Example: JMP 0x1234
0x0100: 0x90 ; JMP instruction
0x0101: 0x12 ; Target address high
0x0102: 0x34 ; Target address low

Circuit Implementation

Address Latch System

Jump Target Storage:
16-bit manual switches → 74F374N latches → Counter parallel inputs
Or: SRAM data → Address assembly logic → Counter parallel inputs

Load Control:
Jump instruction detected → Load enable pulse → All counters load simultaneously

Conditional Jump Logic

Flag Register outputs → AND gates → Conditional load enable
Example for JZ (Jump if Zero):
Zero flag AND JZ instruction → PC load enable

Up/Down Control

Normal execution: U/D = HIGH (count up)
Reverse execution: U/D = LOW (count down)
Control via instruction decode or manual switch

Build Steps

1. Counter Replacement

- Remove existing SN74LS590N counters
- Install four SN74LS190N counters in chain configuration
- Verify basic counting operation (up and down)

- Test rollover behavior at boundaries

2. Parallel Load Implementation

- Connect 16-bit switch bank to counter P0-P3 inputs
- Wire common /PL signal to all counters
- Add manual load pushbutton for testing
- Verify immediate address loading

3. Address Latch Integration

- Install 74F374N latches for jump target storage
- Connect to either manual switches or instruction data path
- Add load control logic for automatic operation
- Test address capture and transfer

4. Instruction Decode Enhancement

- Extend existing MC74F521N comparator logic
- Add detection for jump instruction opcodes (1001-1101)
- Generate appropriate load enable signals
- Implement conditional logic for flag-based jumps

Test Procedures

1. Basic Counter Function Test

Objective: Verify 16-bit counter operation

Manual Test Setup:

```
16x DIP switches → Counter parallel inputs
1x Pushbutton → /PL (parallel load enable)
1x Pushbutton → CP (manual clock)
1x Toggle → U/D (up/down control)
1x Pushbutton → /MR (reset)
```

Test Sequence:

1. Reset counters to 0x0000
2. Clock 10 times in UP mode → Verify 0x000A

3. Switch to DOWN mode, clock 5 times → Verify 0x0005
4. Set switches to 0x1234, pulse load → Verify 0x1234
5. Clock once → Verify 0x1235

2. Jump Instruction Test

Objective: Verify unconditional jump execution

Test Program:

Address	Instruction	Data	Description
0x0000	0x90	-	JMP instruction
0x0001	0x00	-	Target high byte (0x0010)
0x0002	0x10	-	Target low byte
0x0003	0x85	-	Should be skipped
0x0004	0x86	-	Should be skipped
...	...	...	...
0x0010	0x8F	-	Jump target (display 0xF)

Expected Behavior:

- PC starts at 0x0000
- Executes JMP, loads 0x0010 into PC
- Next instruction fetched from 0x0010
- Instructions at 0x0003-0x000F are skipped

3. Conditional Jump Test

Objective: Verify flag-based branching

Test Program:

```
0x0000: 0x1F    ; Load 0xF into Register A
0x0001: 0x21    ; Load 0x1 into Register B
0x0002: 0x30    ; ADD (result = 0x10, no overflow)
0x0003: 0xA0    ; JZ 0x0020 (should not jump, Z=0)
0x0004: 0x00    ; Target high
0x0005: 0x20    ; Target low
0x0006: 0x8A    ; Display 0xA (should execute)
0x0007: 0x1F    ; Load 0xFF into Register A
0x0008: 0x21    ; Load 0x01 into Register B
0x0009: 0x30    ; ADD (result = 0x00, overflow, Z=1)
0x000A: 0xA0    ; JZ 0x0030 (should jump, Z=1)
0x000B: 0x00    ; Target high
0x000C: 0x30    ; Target low
0x000D: 0x8B    ; Should be skipped
...
0x0030: 0x8C    ; Jump target (display 0xC)
```

Expected Sequence: 0xA displayed, then 0xC displayed

4. Loop Implementation Test

Objective: Verify program can loop back to earlier addresses

Simple Loop Program:

```
0x0000: 0x80    ; Display 0x0
0x0001: 0x81    ; Display 0x1
0x0002: 0x82    ; Display 0x2
0x0003: 0x90    ; JMP 0x0000 (loop back)
0x0004: 0x00    ; Target high
0x0005: 0x00    ; Target low
```

Expected Behavior: Continuous sequence 0, 1, 2, 0, 1, 2, ...

Logic Analyzer Validation

Channel Assignment

Channels	Signal	Purpose
0-15	PC output	Current program counter value
16-31	Parallel load inputs	Jump target address
32	/PL signal	Load enable timing
33	U/D control	Count direction
34	Clock signal	Timing reference
35	Reset signal	System state
36-43	Instruction data	Jump instruction detection
44	Zero flag	Conditional jump input
45	Carry flag	Conditional jump input
46-47	Jump enable signals	Control logic verification

Critical Timing Measurements

- **Load Setup Time**: Address stable to /PL active
- **Load Hold Time**: Address stable after /PL inactive
- **Jump Latency**: Instruction decode to PC update
- **Flag Response**: ALU flag change to conditional jump decision

Success Criteria

- [ ] 16-bit counter counts reliably in both directions
- [ ] Parallel load works instantly and accurately
- [ ] Unconditional jumps execute to correct addresses
- [ ] Conditional jumps respond properly to flag states
- [ ] Boundary conditions handled (0x0000 ↔ 0xFFFF)
- [ ] No timing races during jump operations
- [ ] Loop programs execute continuously without errors
- [ ] Jump instruction decode generates proper control signals

Performance Validation

Maximum Jump Frequency

- Determine fastest reliable jump operation rate
- Verify timing margins for back-to-back jumps
- Test rapid conditional branch sequences

Memory Access Patterns

- Verify SRAM access timing during jump operations
- Confirm address setup/hold requirements met
- Test non-sequential memory access reliability

Troubleshooting Guide

Issue: Counter doesn't load parallel data

- Check /PL signal timing and polarity
- Verify parallel input connections
- Confirm counter power supply stability

Issue: Conditional jumps don't work

- Verify flag register outputs
- Check conditional logic gate connections
- Confirm instruction decode for jump opcodes

Issue: Erratic jumping behavior

- Check for timing races in load enable logic
- Verify address setup/hold times
- Add additional decoupling capacitors

Issue: Counter doesn't chain properly

- Verify TC to CP connections between counters
- Check ripple carry timing
- Confirm all counters have same U/D control

Integration Notes for Phase 5

- Document jump instruction timing requirements
- Identify opportunities for subroutine call/return
- Plan stack implementation for nested calls
- Note maximum program size limitations
- Record performance characteristics for optimization

Advanced Features for Future Enhancement

- **Subroutine Stack**: Implement return address storage
- **Relative Jumps**: Add PC-relative addressing mode
- **Interrupt Handling**: Reserve jump vectors for interrupts
- **Breakpoint Support**: Add debug halt capability

Documentation Deliverables

- Complete jump instruction reference
- Timing diagrams for all jump types
- Test program examples demonstrating loops and branches
- Logic analyzer captures of jump operations
- Performance benchmark data
- Integration schematic with jump control logic